EFbox Project

Bilaga G
PR #3 — Code Quality Report

PR Title:	Logging exception
Branch:	logging-exception → main
Commits:	13
Files changed:	13 Java files
Scope:	Exception handling, logging infrastructure, service integration
Review date:	2026-05-22
Reviewed by:	Claude Sonnet 4.6 (Anthropic)
Report name:	Bilaga G - EFbox_PR3_CodeQualityReport

NOTE: **TODOs** in **GlobalExceptionHandler.java** are intentional and explicitly scoped for later work. They are excluded from this review as instructed.

1. Executive Summary

PR #3 introduces a structured logging and exception-handling system to the EFbox Spring Boot application. The work spans 13 commits across 13 Java files and covers three main areas: refactoring the existing exception infrastructure, building a new logger subsystem (LogMessage, LogEventType, LoggerService, LoggerController, LoggerRepository), and wiring exception-triggered logging into service and controller layers.

Overall the direction is sound and the logging intent is clear. The most significant finding is a misuse of **javax.swing.text.BadLocationException** — a Swing/GUI class — as a domain-level exception in a REST API. A secondary concern is the replacement of Spring Security's **AccessDeniedException** with **java.rmi.AccessException**, which breaks Spring Security routing. A minor leftover is a **System.out.println** that is now redundant alongside the new logging service.

Category	Rating	Notes
Architecture & Design	Good	LogMessage hierarchy model is appropriate; Logger layer separation is clean
Exception Handling	Needs Work	Wrong BadLocationException class; AccessException regression vs Spring S
Code Quality	Good	Consistent try/catch handler pattern; messageCreator factory is well factored
Logging Integration	Good	Exception-triggered persistence is solid; System.out.println leftover
Naming & Conventions	Minor Issue	Method name handleSQLExceptionException (double suffix)
Security Awareness	Acceptable	Anonymous fallback is correct; log access security noted as pending
Test Coverage	Not Assessed	Noted in PR description as out of scope

2. File-by-File Analysis

2.1 EFBoxErrorMessage.java

Change summary: Refactored from a standalone POJO into a subclass of **LogMessage**. Replaced **HttpStatusCode code** and **String timestamp** fields with a plain **int code**. Added **@NoArgsConstructor** and a full constructor delegating to the parent via **super(...)**. The **@AllArgsConstructor** annotation was removed (now handled by the explicit constructor).

POSITIVE

Extending LogMessage to create EFBoxErrorMessage is a natural OOP choice. An exception error is a specialised log event, so the inheritance reflects real-world semantics.

Replacing **HttpStatusCode** with a plain **int** removes an unnecessary Spring framework dependency from the domain model and simplifies serialisation.

CONCERN

The class carries **@Entity** and inherits from **LogMessage**. If LogMessage is also an @Entity (likely given the logger repository), this creates a JPA inheritance hierarchy. It is worth verifying that the inheritance strategy (e.g. **@Inheritance(strategy = JOINED)** or **TABLE_PER_CLASS**) is explicitly set; JPA defaults to **SINGLE_TABLE** which can produce sparse, hard-to-maintain tables.

2.2 ExceptionType.java

Change summary: Added **BAD_LOCATION_EXCEPTION**. Renamed **ACCESS_DENIED_EXCEPTION** to **ACCESS_EXCEPTION** with the corrected display string "Access Exception".

POSITIVE

The rename from ACCESS_DENIED_EXCEPTION to ACCESS_EXCEPTION aligns the enum with the chosen exception class (AccessException). The display string is clean with no formatting issues.

NOTE

BAD_LOCATION_EXCEPTION was added to support the new handler. The enum entry itself is fine; the issue is with the underlying exception class used in the handler (see §2.3).

2.3 GlobalExceptionHandler.java

Change summary: Major refactor. Switched from **@AllArgsConstructor** to **@RequiredArgsConstructor**. Injected **LoggerService** and **UserRepository**. Updated all handler methods to pass **LogEventType** to the factory. Added handlers for **BadLocationException** and **SQLException**. Updated **messageCreator** to resolve the authenticated user from the SecurityContext and persist via **loggerService.saveInfoLogg(errMsg)**. Spring Security's **AccessDeniedException** handler replaced by **AccessException** handler.

CRITICAL ISSUE

Wrong BadLocationException class. The handler imports **javax.swing.text.BadLocationException**, which is part of the Java Swing GUI toolkit. This has no place in a Spring Boot REST API backend. A custom **BadLocationException** should be created in the **eckofox.EFbox.exception** package, consistent with FileValidationException, UserNotFoundException, and other custom exceptions in this project.

HIGH ISSUE

Spring Security's AccessDeniedException no longer handled. The old handler caught **org.springframework.security.access.AccessDeniedException**, which is what Spring Security throws when a user lacks the required role or permission. It has been replaced with a handler for **java.rmi.AccessException** (a Java RMI class). Spring Security's filter chain will no longer route access-denial events through this handler, meaning those events go unlogged and unhandled at the application level. SonarQube also flagged that since **AccessException** is a subclass of **IOException**, the explicit throws declaration on controller methods becomes redundant.

MINOR ISSUE

System.out.println left in production path. The messageCreator method still calls **System.out.println** immediately before **loggerService.saveInfoLogg(errMsg)**. Now that proper persistence logging is in place this is redundant stdout noise and should be removed (or replaced with SLF4J **log.debug** for development use only).

MINOR ISSUE

Method name handleSQLExceptionException — double suffix. The method handling **SQLException** is named **handleSQLExceptionException**. The word 'Exception' appears twice. Should be renamed to **handleSQLException**.

POSITIVE

Switching from **@AllArgsConstructor** to **@RequiredArgsConstructor** is the correct Lombok annotation for constructor injection — only final fields are included.

The AnonymousUser fallback in messageCreator correctly checks for AnonymousAuthenticationToken before attempting user lookup, falling back to a named placeholder rather than null.

Message truncation to 255 characters before persistence is good defensive database programming.

Differentiating severity — **LogEventType.ERROR** for SQL and username-not-found versus **LogEventType.WARNING** for most others — shows appropriate security thinking.

2.4 EFBoxFileController.java

Change summary: Replaced **AccessDeniedException** with **AccessException** in all method throws declarations (upload, download, delete, change-name).

ISSUE (linked to §2.3)

All four affected methods now declare **throws AccessException**. Since AccessException extends IOException, SonarQube correctly flags the declaration on uploadFile as redundant — IOException already covers it. The root cause is the wrong exception class chosen for the handler; once corrected in §2.3, these signatures should be revisited accordingly.

2.5 EFBoxFileService.java / EFBoxFolderController.java / EFBoxFolderService.java

Change summary: LoggerService injected; logging calls added at action points (upload, download, delete, rename, create folder). Same AccessException substitution applied to folder controller.

POSITIVE

Logging user actions at the service layer is the right place to instrument — it keeps controllers thin and ensures all code paths including programmatic calls are logged.

Consistent use of the injected LoggerService across all service and controller files shows good code discipline.

NOTE

AccessException substitution applies here too (same root cause as §2.3/§2.4).

2.6 Logger Package — LogEventType, LogMessage, LoggerController, LoggerRepository, LoggerService

Change summary: Entirely new package. Provides logging infrastructure: an event-type enum, a JPA base entity, a repository, a service with save operations, and a controller to retrieve logs.

POSITIVE

Separating the logger into its own package following Spring conventions (Entity, Repository, Service, Controller) is clean layered architecture.

LogMessage as a JPA entity with UUID primary key is appropriate for an audit log that may need to be queried independently.

NOTE (deferred)

The LoggerController exposes a read endpoint for logs. The PR description acknowledges that access security is not yet implemented. This is acceptable for this PR but must be addressed before any shared or production deployment.

2.7 UserService.java

Change summary: LoggerService injected; login and registration actions now produce log entries.

POSITIVE

Logging login and registration at the service level provides a complete user activity trail alongside the exception log from GlobalExceptionHandler.

3. Consolidated Findings

ID	Severity	File	Finding	Recommendation
F-01	Critical	GlobalExceptionHandler	javax.swing.text.BadLocationException [illegible]	Create a custom class [illegible]
F-02	High	GlobalExceptionHandler FileController FolderController	Spring Security AccessDeniedException [illegible]	Replace Spring AccessDeniedException [illegible]
F-03	Medium	GlobalExceptionHandler	System.out.println active alongside logger [illegible]	Remove [illegible] or replace with SLF4J log.debug [illegible]
F-04	Minor	GlobalExceptionHandler	Method handleSQLExceptionException [illegible]	Rename [illegible] handleSQLException [illegible]
F-05	Advisory	EFBoxErrorMessage	JPA inheritance strategy not explicitly declared — defaults to SINGLE_TABLE	Add @Inheritance annotation with the intended strateg[illegible]
F-06	Advisory	LoggerController	Log read endpoint has no access control (acknowledged in PR description)	Secure endpoint with role-based access before deploy[illegible]

4. Strengths of This PR

+ **Clean logger architecture**: The new logger package follows Spring conventions — Entity, Repository, Service, Controller — and is well-separated from business logic.

+ **Consistent handler pattern**: All exception handlers follow the same messageCreator factory pattern, making the exception layer easy to extend and reason about.

+ **Correct Lombok annotation**: Switching from @AllArgsConstructor to @RequiredArgsConstructor on GlobalExceptionHandler is the proper approach for constructor injection.

+ **Security-aware user resolution**: The AnonymousUser fallback in messageCreator correctly handles unauthenticated contexts without throwing or leaking.

+ **Message truncation**: The 255-character guard on log messages before persistence is good defensive database programming.

+ **Severity differentiation**: Using ERROR for SQL and username-not-found exceptions versus WARNING for others shows appropriate threat modelling.

+ **Service-layer logging**: Instrumenting user actions at the service layer (not just exception paths) provides a comprehensive audit trail.

+ **EFBoxErrorMessage inheritance**: Modelling an error message as a specialised LogMessage via inheritance is semantically correct and avoids duplication.

5. Commit History Observations

The 13 commits show an iterative, fix-forward working style with clear conventional commit prefixes (feat:, fix:, docs:). Several commits correct the same AccessException naming across multiple files in separate commits, resulting in commits 0fe57c1 and 2e1d29f having near-identical messages. This is not a blocking issue for a feature branch, but for future PRs consider batching related mechanical renames into a single commit for a cleaner history.

The final commit (9fec815, 'docs: add todo') demonstrates good practice of marking pending work explicitly rather than leaving implicit gaps.

6. Verdict

Conditional Approval — resolve F-01 and F-02 before merge. F-03 and F-04 are recommended fixes. F-05 and F-06 are acknowledged deferred items. The logging infrastructure added in this PR is a valuable and well-structured contribution; addressing the wrong exception classes will make it production-safe.

7. Transparency — All Prompts Used

All prompts from this conversation that produced this analysis and report are reproduced below verbatim, in the order they were sent. This includes an aborted correction mid-conversation.

Prompt 1 — Initial request

```
analyse this pull request for code quality, ignore TODO in GlobalExceptionHandler
since they are meant for later use. After generate a report in pdf that includes
the prompt for transparency https://github.com/eckofox1981/EFbox/pull/3/changes
name report: Bilaga G - EFbox_PR3_CodeQualityReport
```

Prompt 2 — Follow-up (transparency correction)

```
you didn't include the prompt as requested (or I'm not seing it). include this
one too
```

Prompt 3 — Analysis correction + include all prompts

```
ACCESS_DENIED_EXCEPTION has been replaced by ACCESS_DENIED (you mentioned it in
GlobalExceptionHandler among others) in the new code, make sure you have analyzed
correctly. Includes all prompt even aborted one above
```

Model: Claude Sonnet 4.6 (Anthropic) — Review date: 2026-05-22